

EXPERIMENT 22:

22. Construct a C program to implement best fit algorithm of memory management.

PSEUDO CODE :

Start

Input memory blocks

Input process sizes

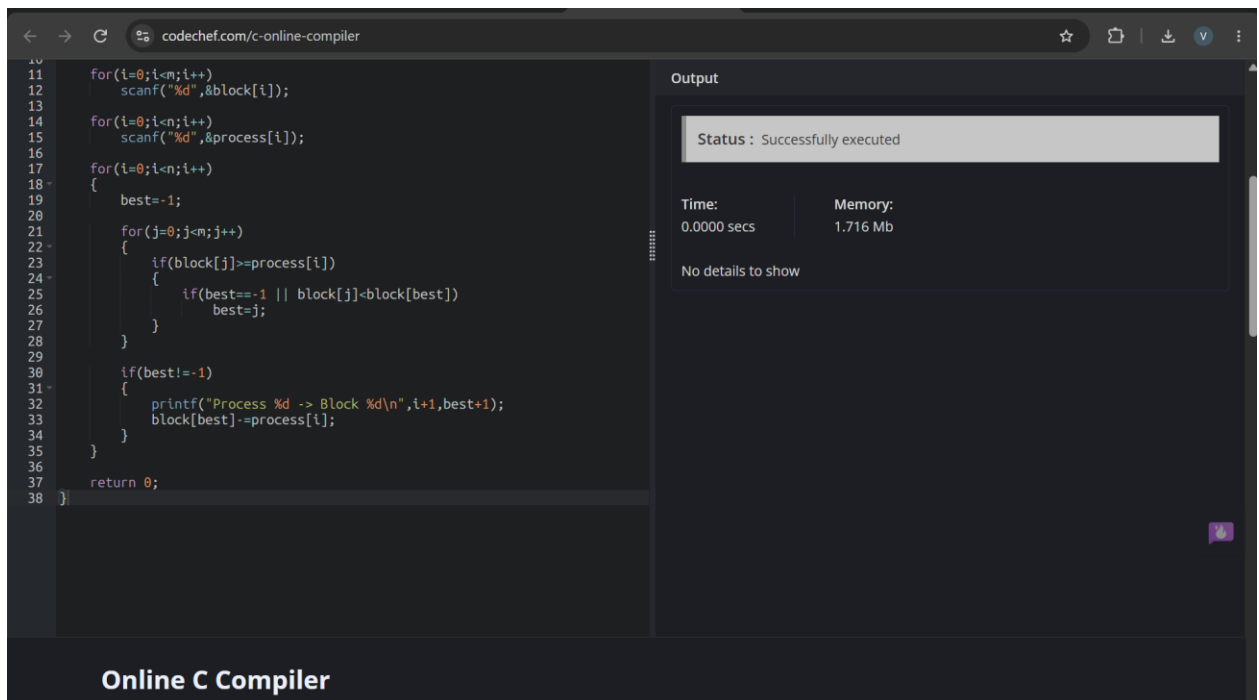
For each process

 Find smallest suitable block

 Allocate process

Display allocation

Stop

A screenshot of a web browser displaying the CodeChef online C compiler. The browser's address bar shows 'codechef.com/c-online-compiler'. The main area is split into two panels. The left panel contains C code for finding the maximum value in an array 'block' by comparing it with elements in 'process'. The code includes loops for input, a nested loop for comparison, and a printf statement for output. The right panel, titled 'Output', shows a status message 'Status : Successfully executed', execution time '0.0000 secs', and memory usage '1.716 Mb'. The browser's bottom status bar says 'Online C Compiler'.

```
10
11 for(i=0;i<m;i++)
12     scanf("%d",&block[i]);
13
14 for(i=0;i<n;i++)
15     scanf("%d",&process[i]);
16
17 for(i=0;i<n;i++)
18 {
19     best=-1;
20     for(j=0;j<m;j++)
21     {
22         if(block[j]>=process[i])
23         {
24             if(best==-1 || block[j]<block[best])
25                 best=j;
26         }
27     }
28
29     if(best!=-1)
30     {
31         printf("Process %d -> Block %d\n",i+1,best+1);
32         block[best]=process[i];
33     }
34 }
35
36 return 0;
37 }
38 }
```

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	1.716 Mb

No details to show

Online C Compiler

CODE:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int
    block[10],process[10];
```

```
    int m,n,i,j,best;
```

```
    scanf("%d",&m);
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<m;i++)
```

```
    scanf("%d",&block[i]);
```

```

    for(i=0;i<n;i++)

scanf("%d",&process[i])
;

    for(i=0;i<n;i++)

    {

        best=-1;

        for(j=0;j<m;j++)

        {

            if(block[j]>=process[i])

            {

                if(best==-1 ||
block[j]<block[best])

                    best=j;

            }

        }

        if(best!=-1)

        {

            printf("Process
%d -> Block
%d\n",i+1,best+1);

            block[best]-
=process[i];

```

```
}
```

```
}
```

```
return 0;
```

```
}
```